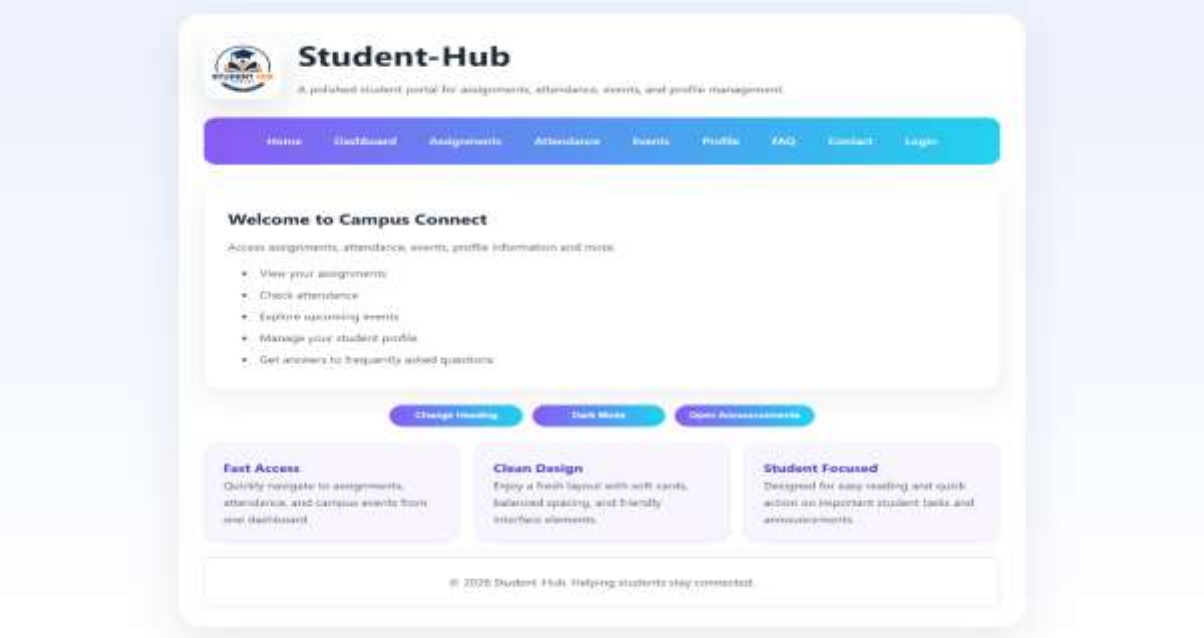
PRACTICAL – 10

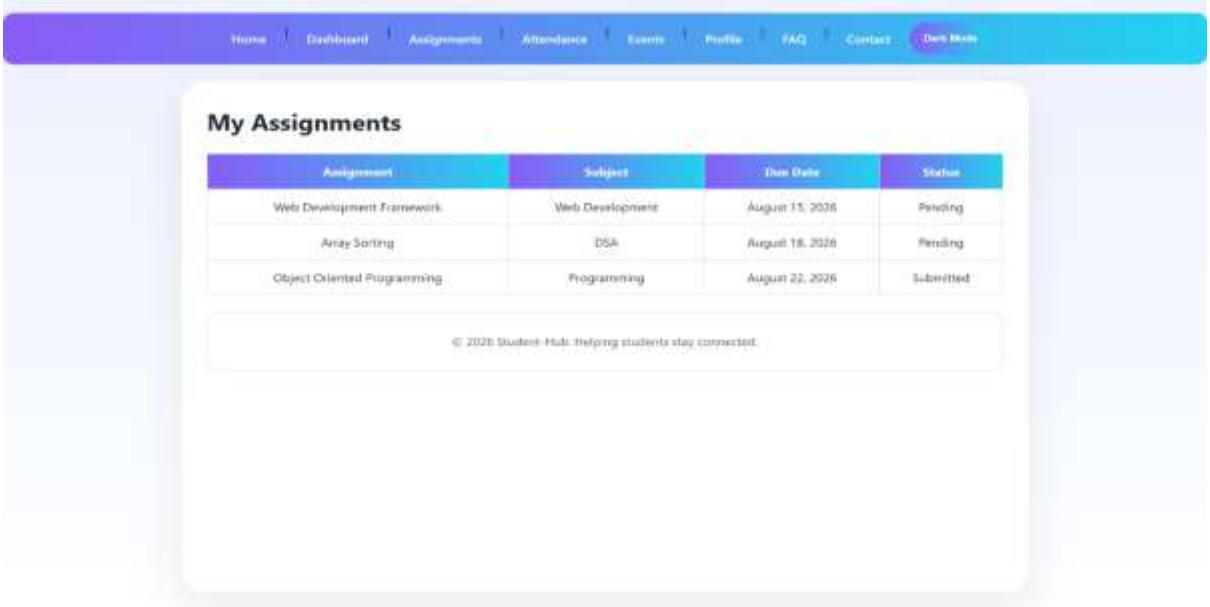
Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

Output:

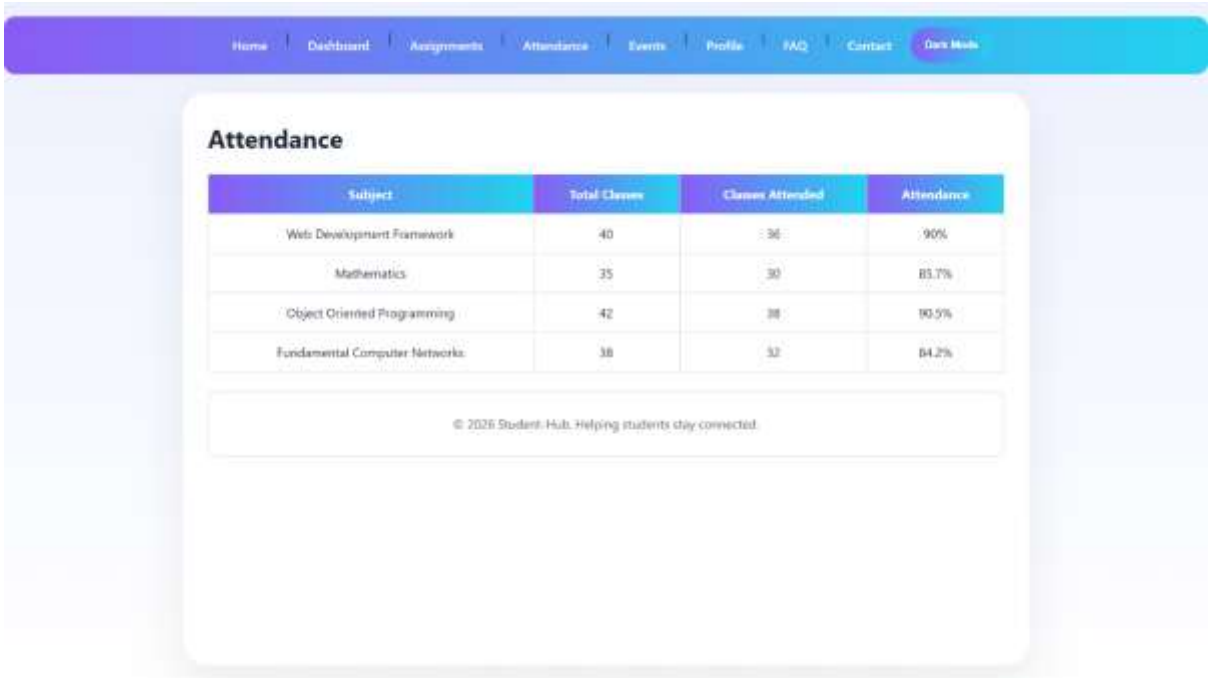
Home Page:



Assignment Page:

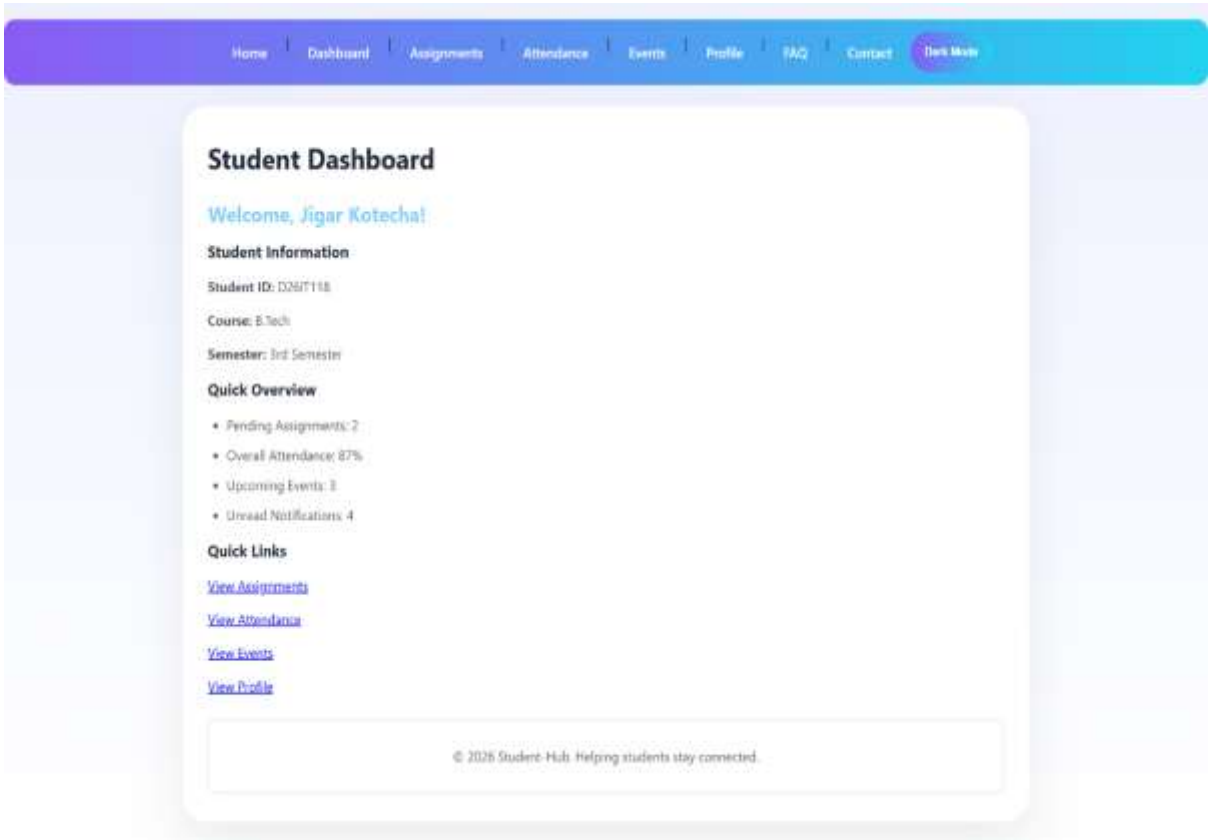


Attendance Page :

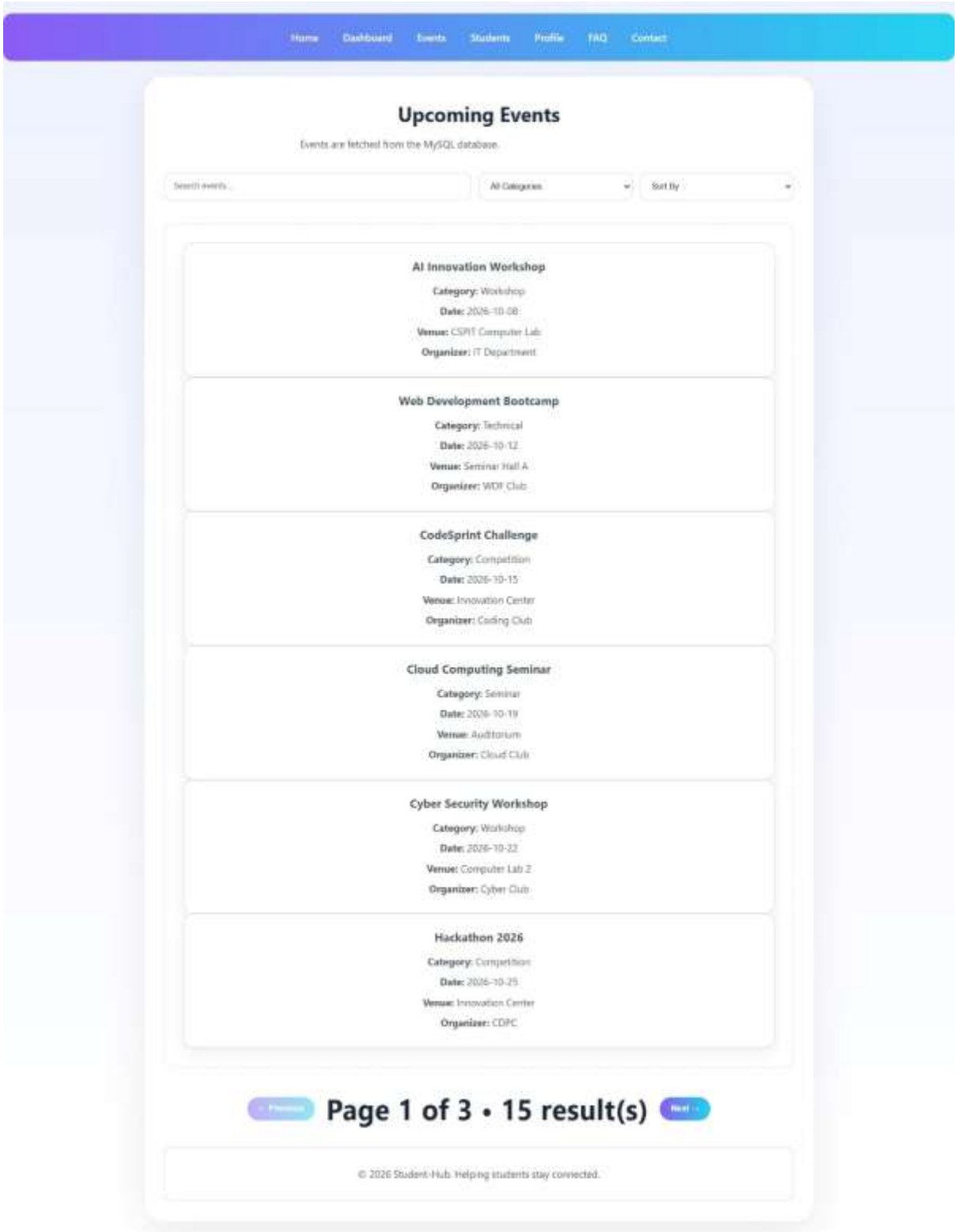


---

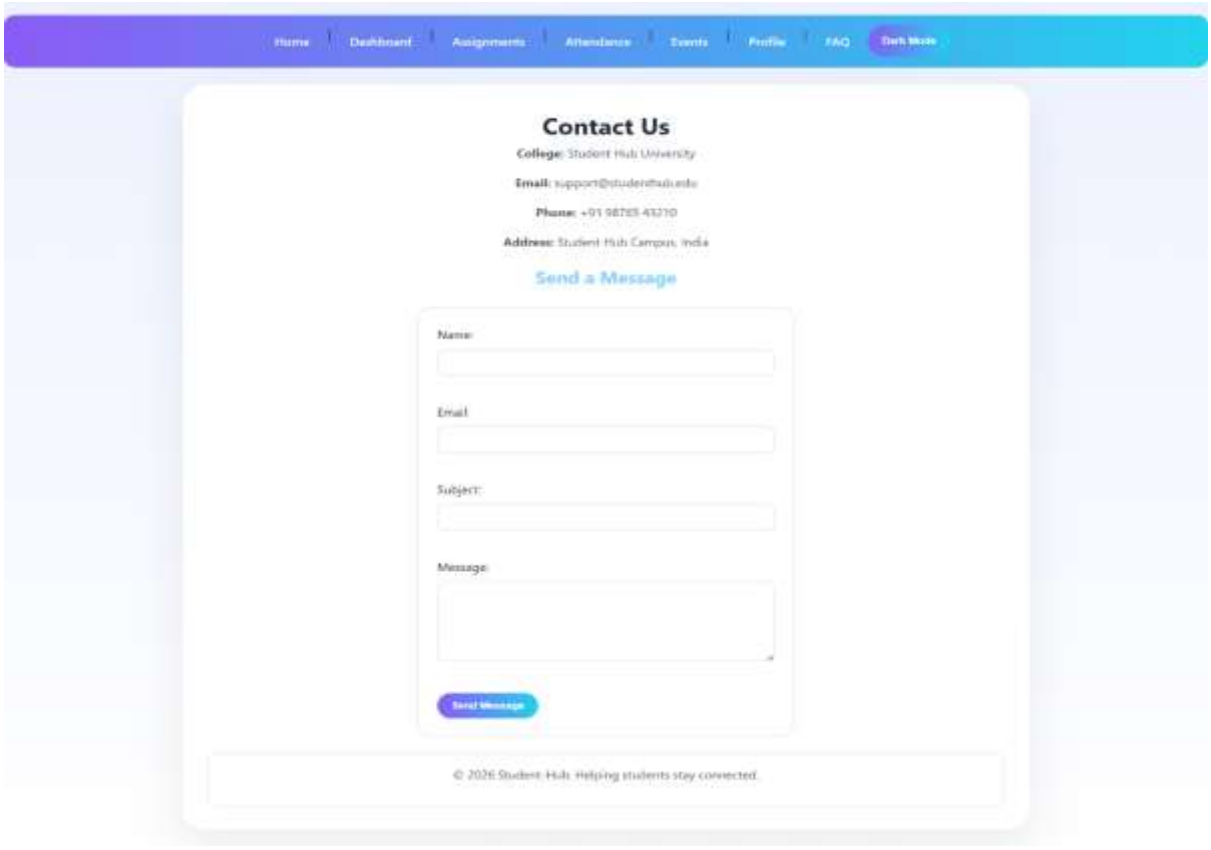
dashboard page :



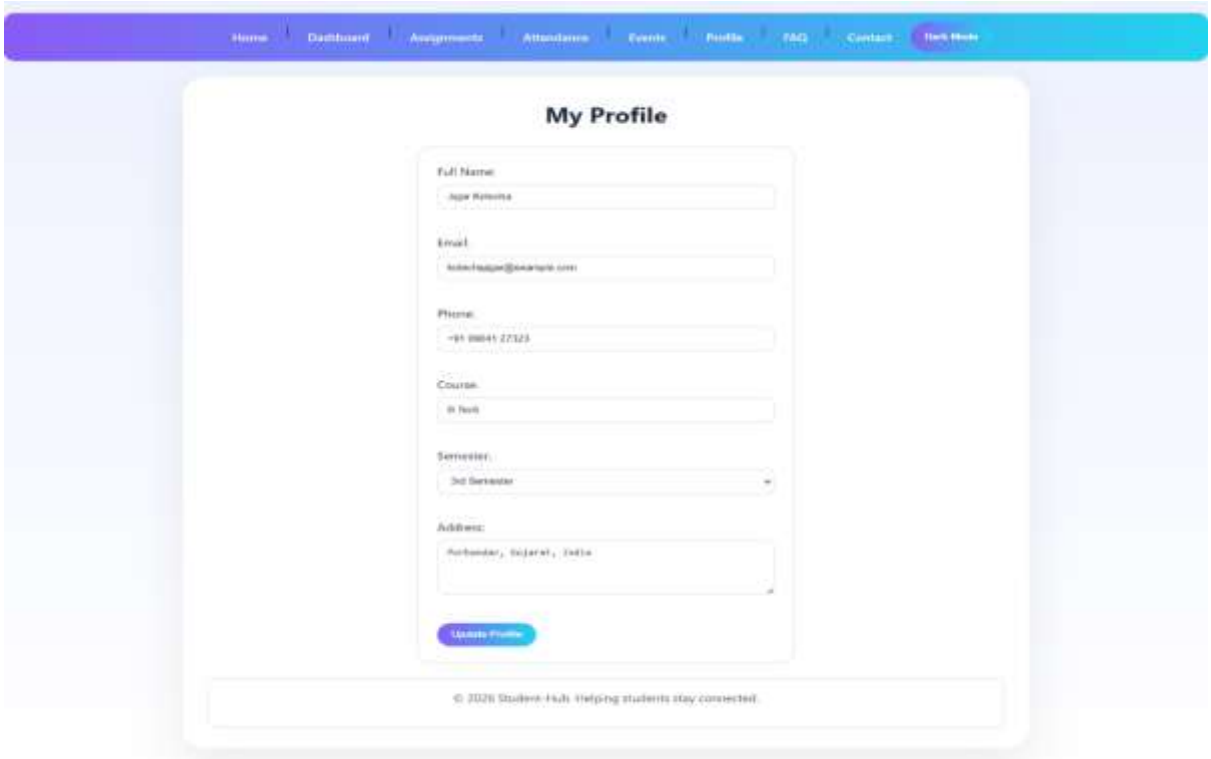
Event Page :


Home
Dashboard
Events
Students
Profile
FAQ
Contact
Upcoming Events
Events are fetched from the MySQL database.
Search events...
All Categories
Sort By
AI Innovation Workshop
Category: Workshop
Date: 2026-10-08
Venue: CSPIT Computer Lab
Organizer: IT Department
Web Development Bootcamp
Category: Technical
Date: 2026-10-12
Venue: Seminar Hall A
Organizer: WDF Club
CodeSprint Challenge
Category: Competition
Date: 2026-10-15
Venue: Innovation Center
Organizer: Coding Club
Cloud Computing Seminar
Category: Seminar
Date: 2026-10-19
Venue: Auditorium
Organizer: Cloud Club
Cyber Security Workshop
Category: Workshop
Date: 2026-10-22
Venue: Computer Lab 2
Organizer: Cyber Club
Hackathon 2026
Category: Competition
Date: 2026-10-25
Venue: Innovation Center
Organizer: CDPC
Page 1 of 3 • 15 result(s)
© 2026 Student Hub. Helping students stay connected.

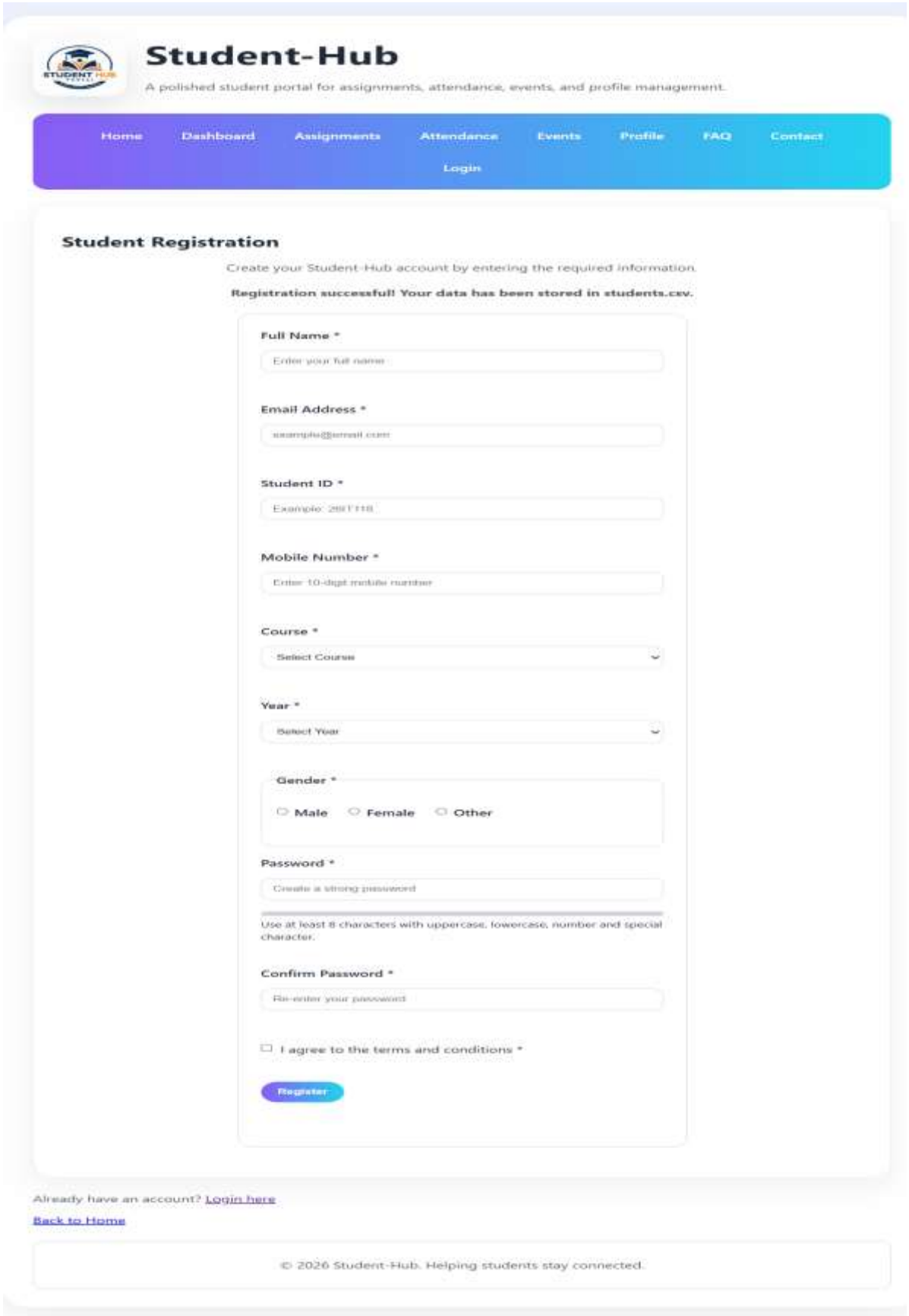
Contact page :



Profile page :



Register page :

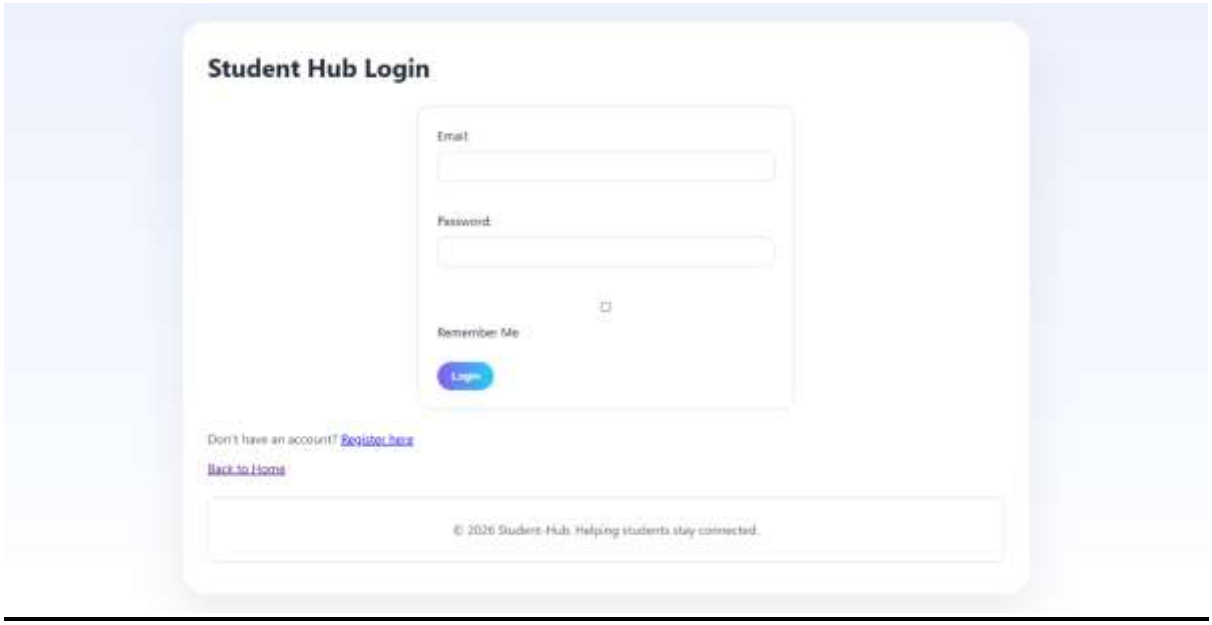


Student.csv :

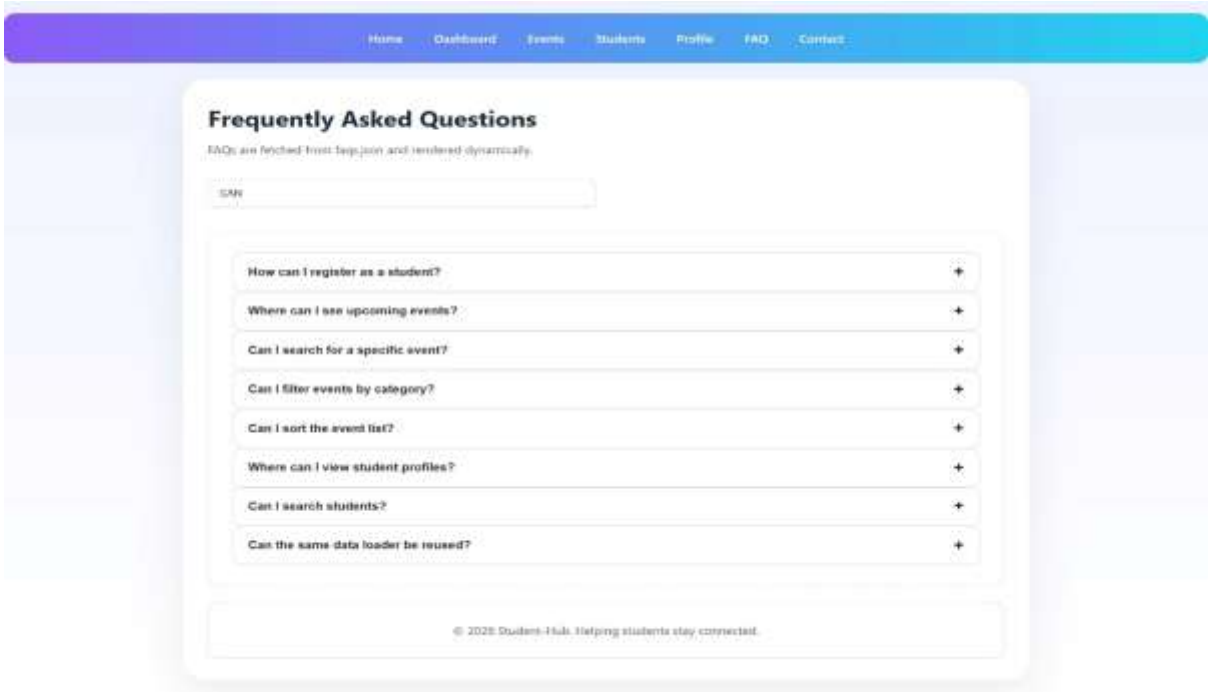


Full Name	Email	Student ID	Mobile	Course	Year	Gender
IKGAR KOT	d26it118@	D26IT118	9.9E+09	btech		2 male

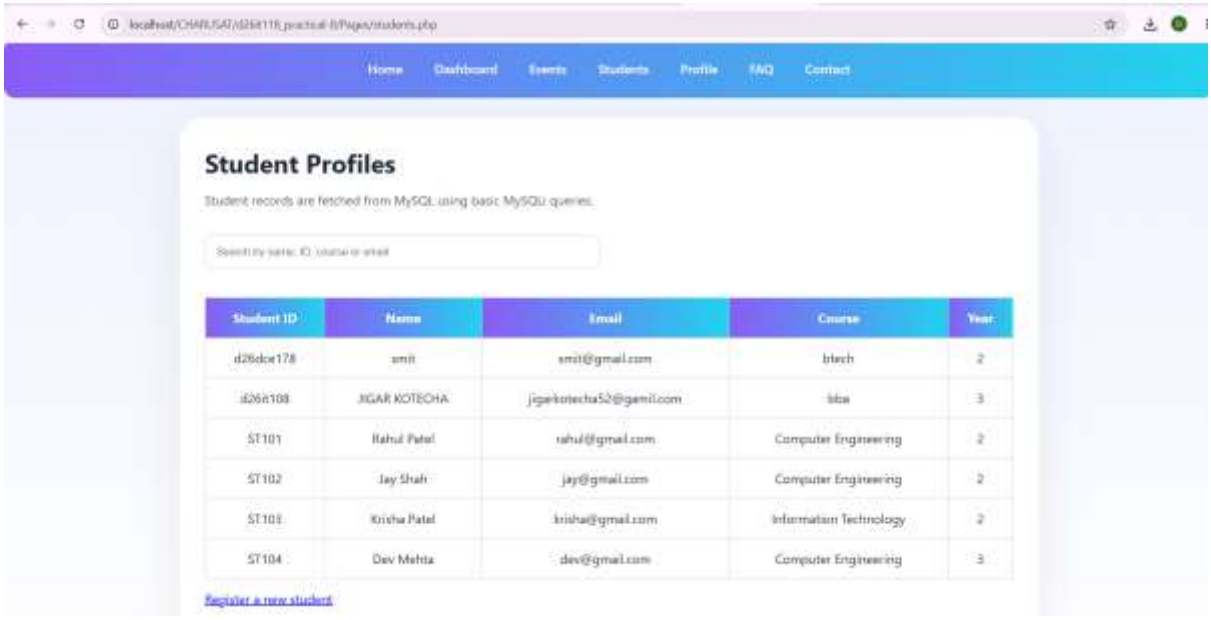
Login page :



Faqs page :

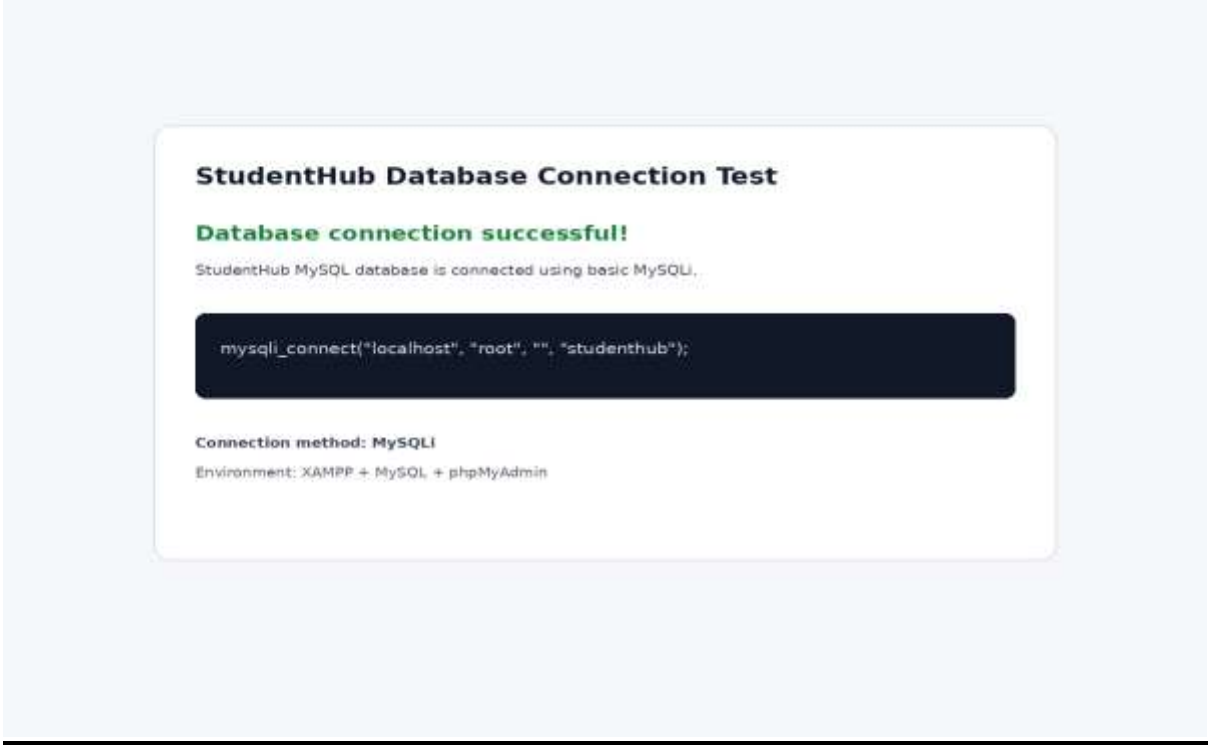


Students page :

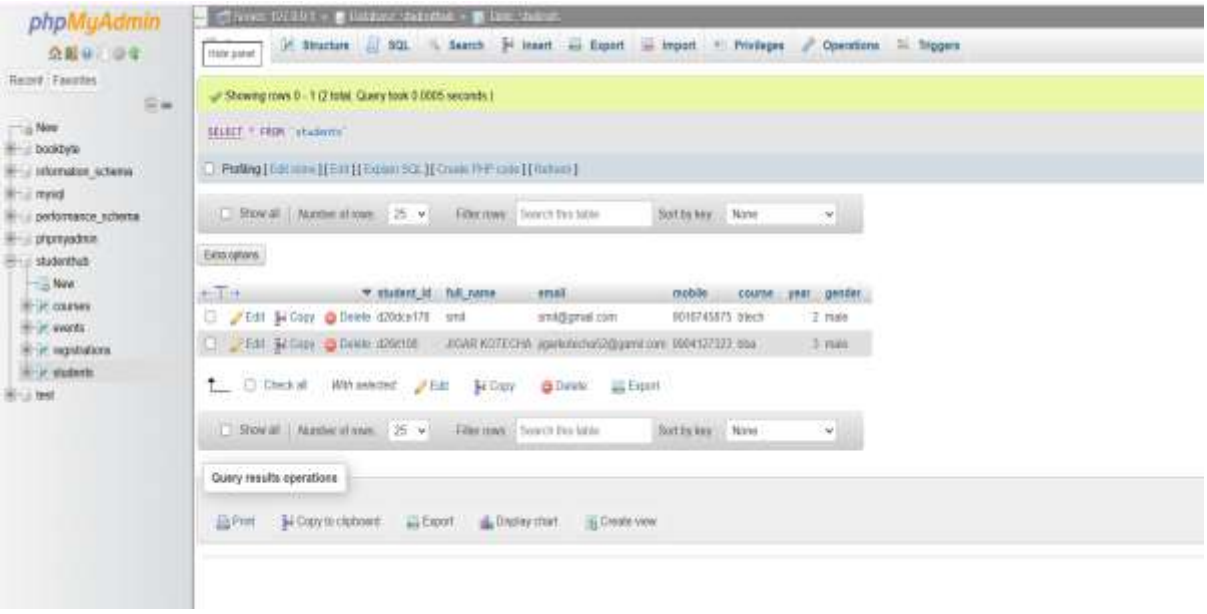


Database :

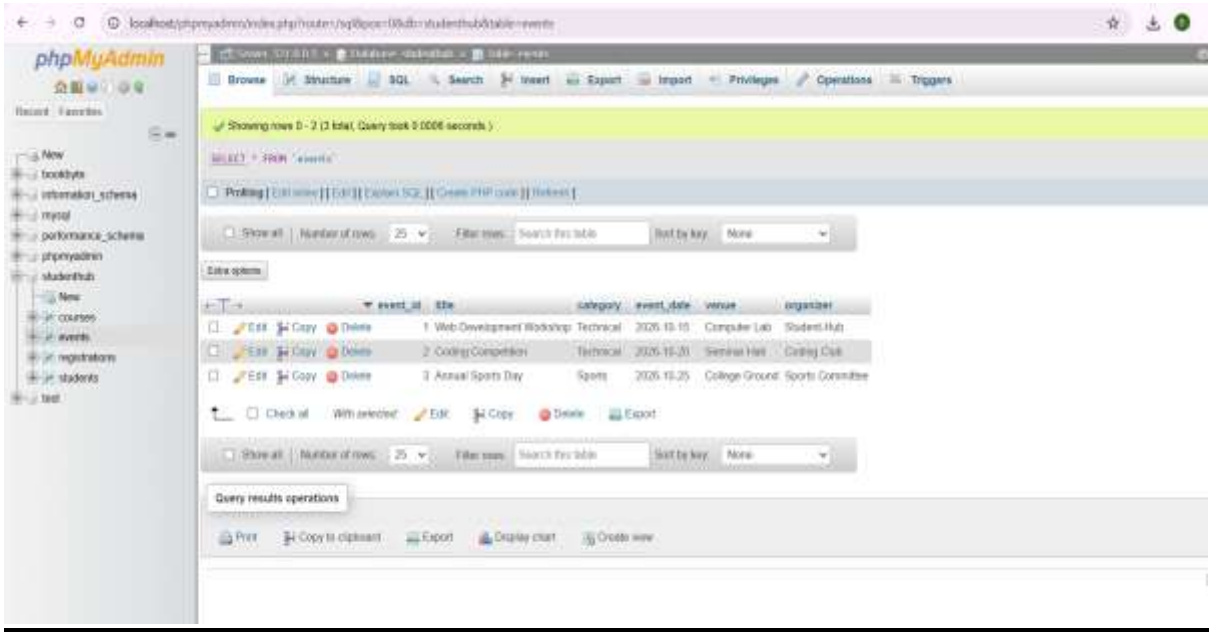
Connection :



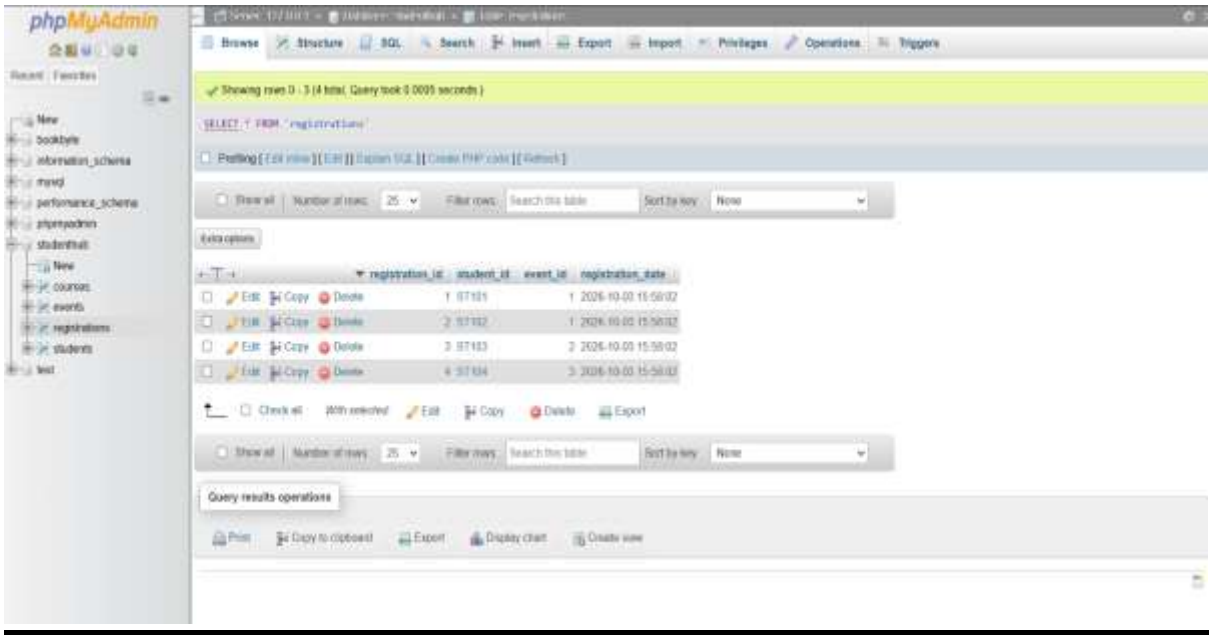
Student table :



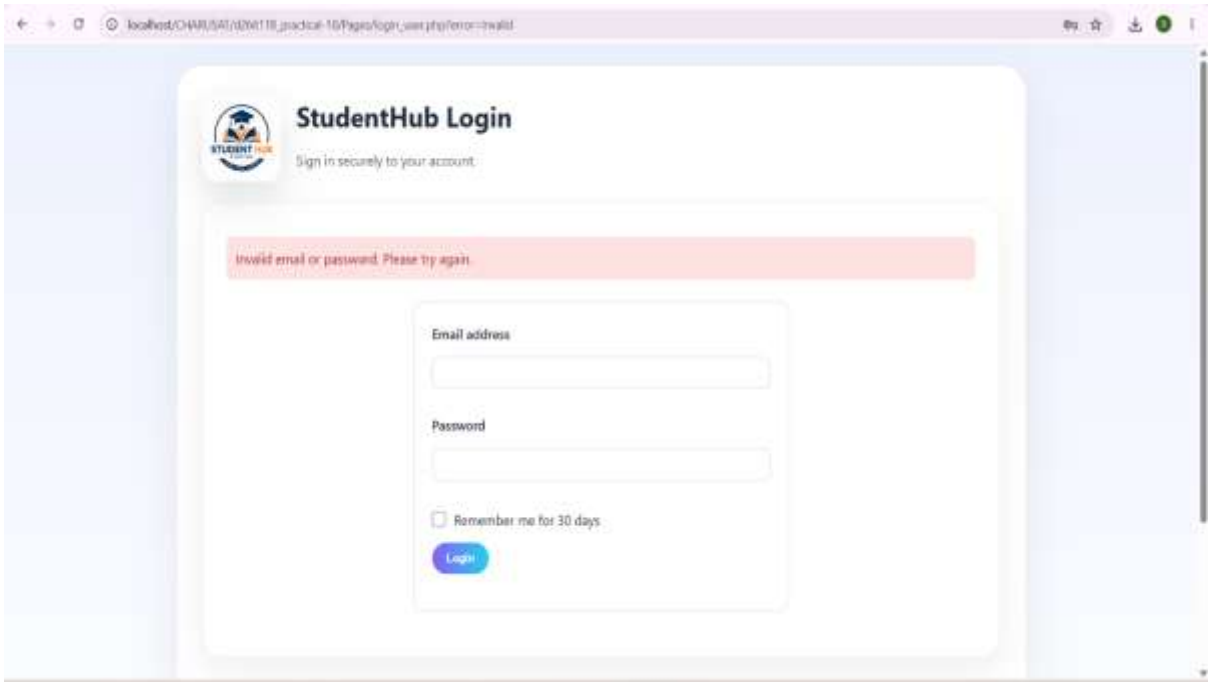
Events table :



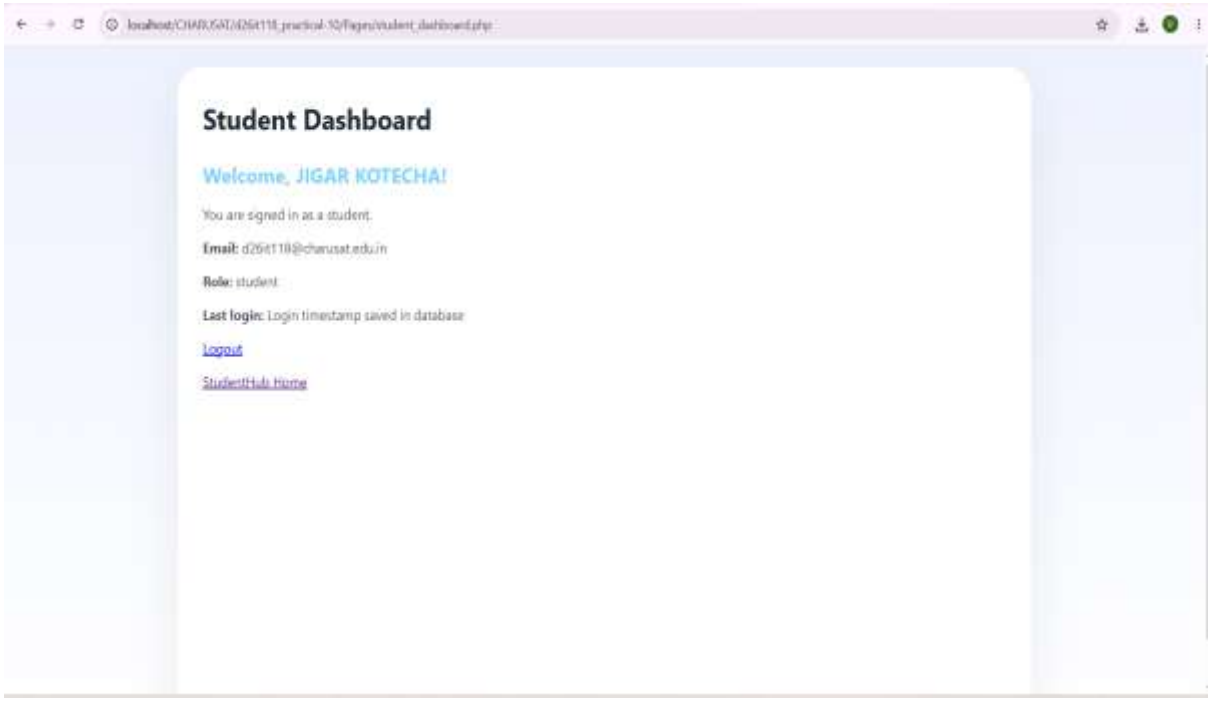
Registrations table :



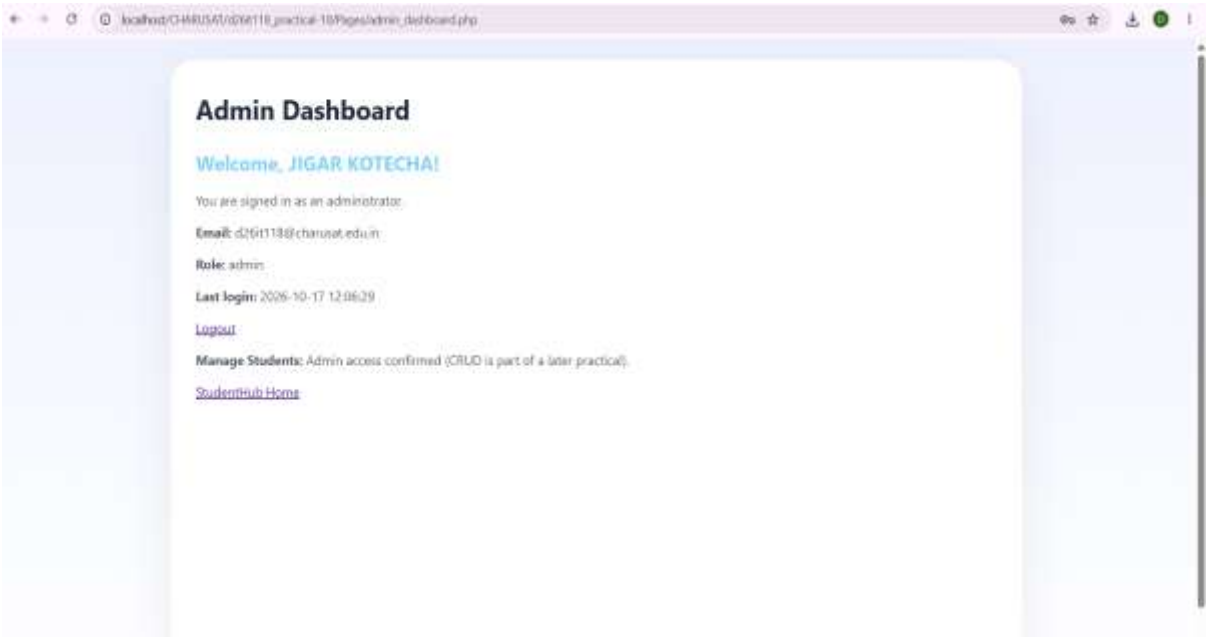
Login page error :



Student_Dashboard :



Admin Dashboard :



Question & Answer:

1. Are sessions started and destroyed securely?

Sessions are started using session_start() to maintain user login information. The session ID is regenerated after successful login to improve security. During logout, session variables are cleared and the session is destroyed using session_destroy(). These steps help protect user information and prevent unauthorized access to the account.

2. Is password_verify() used correctly?

The password_verify() function checks the entered password against the hashed password stored in the database. If the password matches, the user is allowed to log in successfully. If the password does not match, access is denied. This method avoids storing passwords in plain text and improves login security.

3. Is unauthorized dashboard access blocked?

The dashboard checks whether the user has an active and valid login session before allowing access. If a user tries to open the dashboard without logging in, the system denies access. The unauthorized user is redirected to the login page. This ensures that only authenticated users can access protected dashboard pages.

4. Is session timeout implemented?

The system tracks the user's last activity time using a session variable. If the user remains inactive for longer than the specified time, the session expires. The session information is cleared, and the user must log in again. This feature helps protect user accounts when a computer is left unattended.

5. Are admin and student users redirected to appropriate dashboards?

After successful login, the system checks the role assigned to the user. If the user has an admin role, they are redirected to the admin dashboard. If the user has a student role, they are redirected to the student dashboard. This ensures that users access the appropriate pages and functionality according to their roles.

Conclusion / Learning Outcome:

In this practical, we learned to implement a secure login and logout system using PHP and MySQL. We used PHP sessions to maintain user authentication and password_verify() to verify hashed passwords securely. We regenerated the session ID after login to improve security. We restricted unauthorized dashboard access and implemented role-based redirection for admin and student users. We also learned to implement session timeout to protect inactive sessions. The remember-me token feature can provide secure persistent login when implemented using hashed tokens. Overall, this practical improved our understanding of authentication, session management, cookies, and role-based access control in secure web applications.

Githublink :- https://github.com/d26it118-png/Student-Hub.git